

Lab1实验报告

思考题

Thinking 1.1

objdump参数解析:

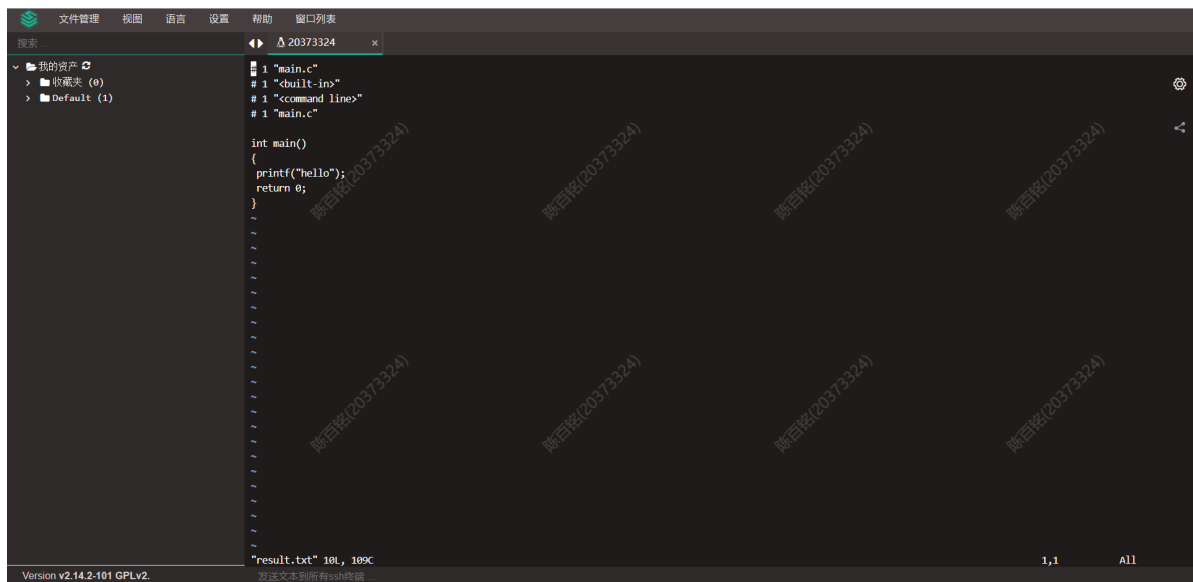
-D disassemble-all 全部反汇编

[-S|--source]

[--source-comment[=text]]

重复教程步骤如下:

/OSLAB/compiler/usr/bin/mips_4KC-gcc -E main.c >result.txt命令后得到



/OSLAB/compiler/usr/bin/mips_4KC-gcc -c main.c 进行编译

/OSLAB/compiler/usr/bin/mips_4KC-objdump -DS main.o >result.txt 进行反汇编后得到result.txt结果

```
main.o:      file format elf32-tradbigmips
```

```
Disassembly of section .text:
```

```
00000000 <main>:
 0: 3c1c0000    lui gp,0x0
 4: 279c0000    addiu gp,gp,0
 8: 0399e021    addu  gp,gp,t9
 c: 27bdf0e0    addiu sp,sp,-32
10: afbf001c    sw   ra,28(sp)
14: afbe0018    sw   s8,24(sp)
18: 03a0f021    move s8,sp
1c: afbc0010    sw   gp,16(sp)
20: 8f820000    lw   v0,0(gp)
24: 24440000    addiu a0,v0,0
28: 8f990000    lw   t9,0(gp)
2c: 0320f809    jalr t9
```

```

30: 00000000    nop
34: 8fdc0010    lw  gp,16(s8)
38: 00001021    move    v0,zero
3c: 03c0e821    move    sp,s8
40: 8fbf001c    lw  ra,28(sp)
44: 8fbe0018    lw  s8,24(sp)
48: 27bd0020    addiu   sp,sp,32
4c: 03e00008    jr  ra
50: 00000000    nop

```

...

Disassembly of section .reginfo:

00000000 <.reginfo>:

```

0: f2000014    0xf2000014

```

...

Disassembly of section .pdr:

00000000 <.pdr>:

```

0: 00000000    nop
4: c0000000    ll  zero,0(zero)
8: ffffffff    sdc3    $31,-4(ra)

```

...

```

14: 00000020    add zero,zero,zero
18: 0000001e    0x1e
1c: 0000001f    0x1f

```

Disassembly of section .rodata:

00000000 <.rodata>:

```

0: 68656c6c    0x68656c6c
4: 6f000000    0x6f000000

```

...

Disassembly of section .comment:

00000000 <.comment>:

```

0: 00474343    0x474343
4: 3a202847    xori    zero,s1,0x2847
8: 4e552920    c3  0x552920
c: 342e302e    ori  t6,at,0x302e
10: 30202844    andi    zero,at,0x2844
14: 454e5820    0x454e5820
18: 454c444b    0x454c444b
1c: 20342e31    addi    s4,at,11825
20: 20342e30    addi    s4,at,11824
24: 2e302900    sltiu   s0,s1,10496

```

接下来在用mips_4KC-gcc进行正常编译时报错:

```

/OSLAB/compiler/usr/bin/./lib/gcc/mips-linux/4.0.0/./.././../mips-linux/bin/ld: crt1.o: No such file:
No such file or directory
collect2: ld returned 1 exit status

```

Thinking 1.2

用 `readelf -h` 解析vmlinux文件, 得到结果如下:

ELF Header:

```

Magic:   7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00

```

```

Class: ELF32
Data: 2's complement, big endian
Version: 1 (current)
OS/ABI: UNIX - System V
ABI Version: 0
Type: EXEC (Executable file)
Machine: MIPS R3000
Version: 0x1
Entry point address: 0x80010000
Start of program headers: 52 (bytes into file)
Start of section headers: 37916 (bytes into file)
Flags: 0x1001, noreorder, o32, mips1
Size of this header: 52 (bytes)
Size of program headers: 32 (bytes)
Number of program headers: 2
Size of section headers: 40 (bytes)
Number of section headers: 14
Section header string table index: 11

```

发现vmlinux为大端存储，而readelf只能对小端存储文件进行解析。

Thinking 1.3

Bootloader的第一部分运行在存放bootloader的存储设备上，为stage 2准备RAM空间并设置堆栈，而stage 2运行在RAM中，此时有足够的运行环境用C语言实现功能，所以可以正确跳转至内存布局的内核地址。

Thinking 1.4

可以将每个程序段的加载的起始地址都设为页对齐。

Thinking 1.5

内核入口在0x00000000，main函数在0x80001000，通过jal跳转进入main，跨文件调用时现在栈上保存当前数据的值，再jal跳转至需要调用的函数处。

Thinking 1.6

mtc0 zero, CP0_STATUS 将cp0寄存器置零，使中断功能失效。

mfc0 t0, CP0_CONFIG 将cp0的第一位和第三位置0，使看门狗失效。

实验难点

Exercise 1.1

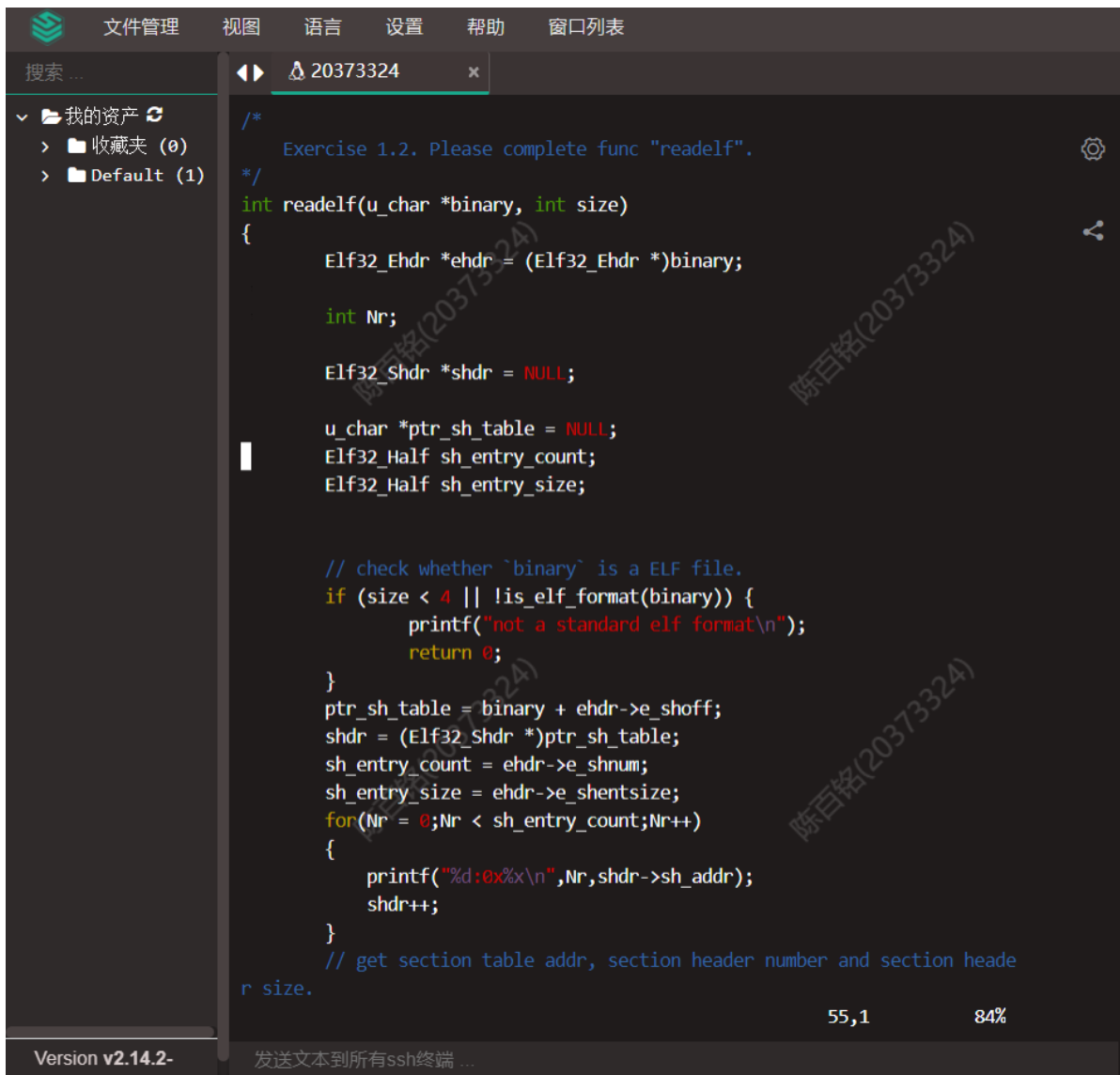
```

CROSS_COMPILE := /OSLAB/compiler/usr/bin/mips_4KC-
CC             := $(CROSS_COMPILE)gcc
CFLAGS        := -O -G 0 -mno-abicalls -fno-builtin -Wa,-xgot -Wall -fPIC
               -march=r3000
LD             := $(CROSS_COMPILE)ld
~
~

```

注意模仿交叉编译器的地址引用方式，较简单。

Exercise 1.2



```
/*  
Exercise 1.2. Please complete func "readelf".  
*/  
int readelf(u_char *binary, int size)  
{  
    Elf32_Ehdr *ehdr = (Elf32_Ehdr *)binary;  
  
    int Nr;  
  
    Elf32_Shdr *shdr = NULL;  
  
    u_char *ptr_sh_table = NULL;  
    Elf32_Half sh_entry_count;  
    Elf32_Half sh_entry_size;  
  
    // check whether `binary` is a ELF file.  
    if (size < 4 || !is_elf_format(binary)) {  
        printf("not a standard elf format\n");  
        return 0;  
    }  
    ptr_sh_table = binary + ehdr->e_shoff;  
    shdr = (Elf32_Shdr *)ptr_sh_table;  
    sh_entry_count = ehdr->e_shnum;  
    sh_entry_size = ehdr->e_shentsize;  
    for(Nr = 0; Nr < sh_entry_count; Nr++)  
    {  
        printf("%d:0x%x\n", Nr, shdr->sh_addr);  
        shdr++;  
    }  
    // get section table addr, section header number and section header size.  
}
```

55,1 84%

Version v2.14.2- 发送文本到所有ssh终端 ...

主要难点在于理解elf文件内容以及结构体变量的含义，理解后注意指针的运算如何指向每个section，有一定理解难度。

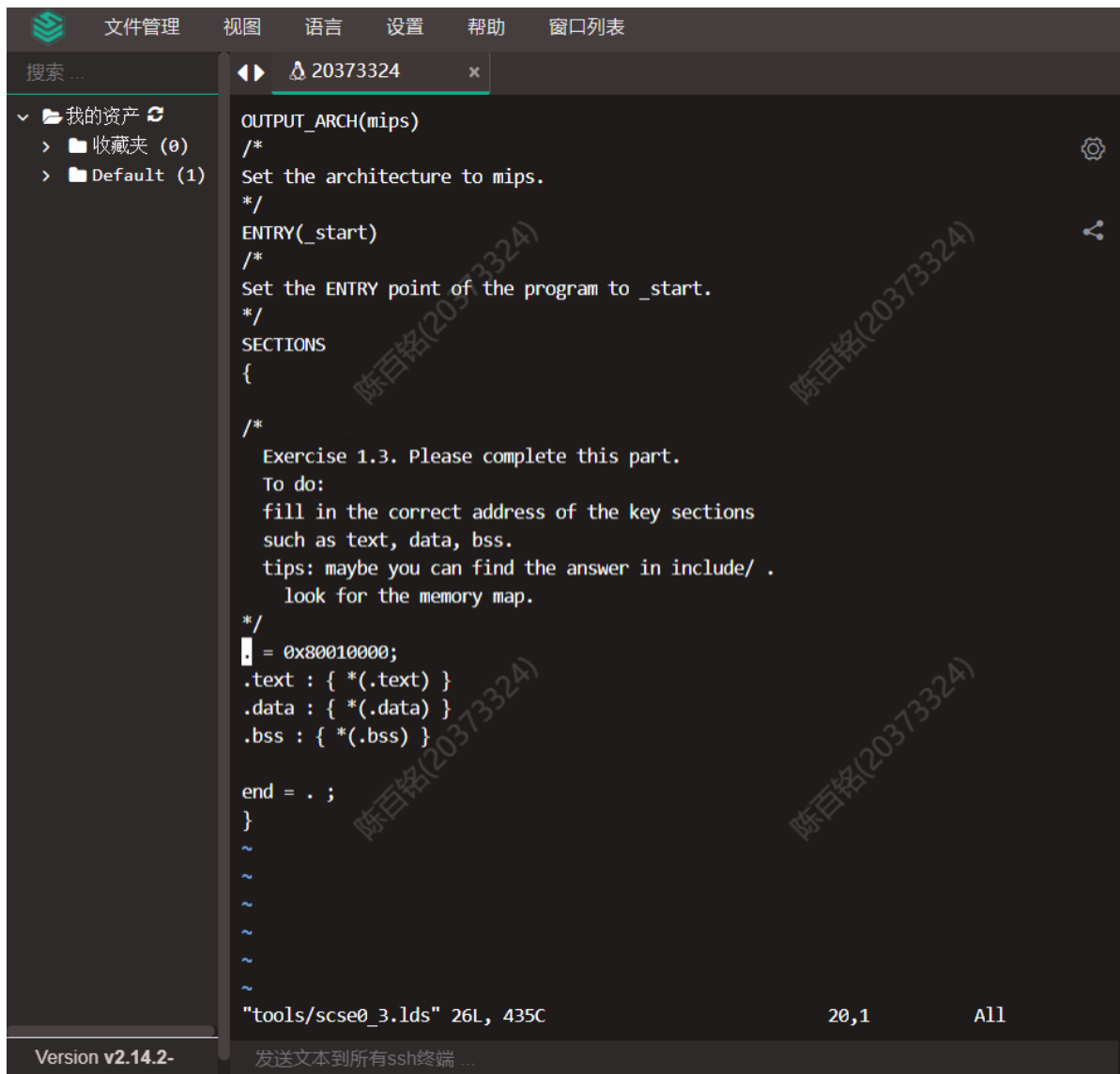
Exercise 1.3

```

/*
o 4G -----> +-----0x100000000
o               |      ...      | kseg3
o               +-----0xe000 0000
o               |      ...      | kseg2
o               +-----0xc000 0000
o               | Interrupts & Exception | kseg1
o               +-----0xa000 0000
o               | Invalid memory | /\
o               +-----Physics Memory Max
o               |      ...      | kseg0
o VPT, KSTACKTOP-----> +-----0x8040 0000-----end
o               |      Kernel Stack      | | KSTKSIZE      /\
o               +-----+-----+
o               |      Kernel Text      | | | PDMAP
o KERNBASE -----> +-----0x8001 0000 |
o               | Interrupts & Exception | \\/ \\/
o ULIM -----> +-----0x8000 0000-----+
o               |      User VPT      | PDMAP      /\
o UVPT -----> +-----0x7fc0 0000 |
o               |      PAGES      | PDMAP
o UPAGES -----> +-----0x7f80 0000 |
o               |      ENVS      | PDMAP
o UTOP, UENVS -----> +-----0x7f40 0000 |
o UXSTACKTOP -/ |      user exception stack | BY2PG
o               +-----0x7f3f f000 |
o               | Invalid memory | BY2PG
o USTACKTOP -----> +-----0x7f3f e000 |
o               |      normal user stack | BY2PG
o               +-----0x7f3f d000 |
a               |
a               ~~~~~
a               .
a               .
a               kuseg

```

这题需要仔细阅读指导书，找到内核地址KERNBASE就很容易了



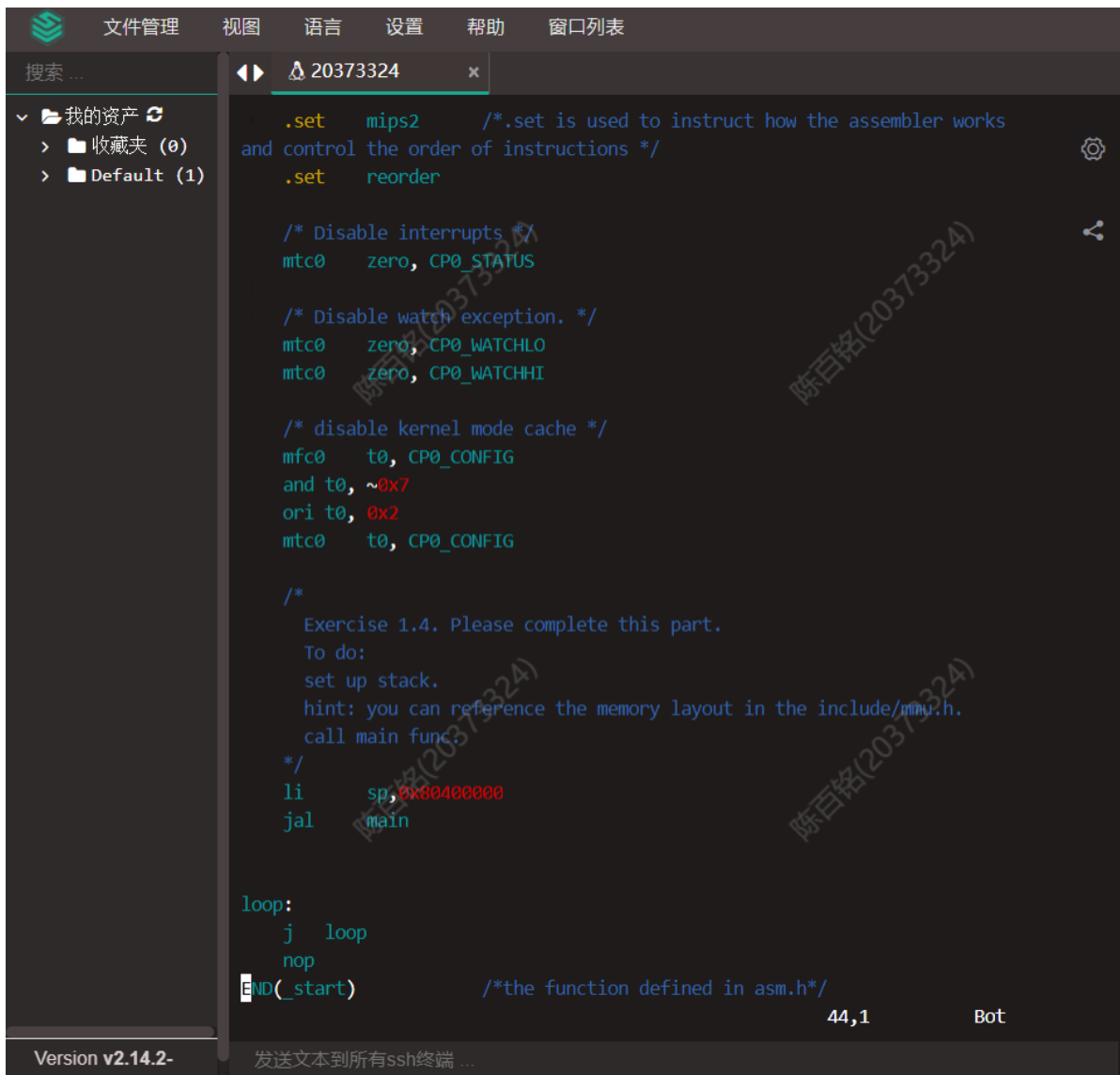
```
OUTPUT_ARCH(mips)
/*
Set the architecture to mips.
*/
ENTRY(_start)
/*
Set the ENTRY point of the program to _start.
*/
SECTIONS
{
/*
Exercise 1.3. Please complete this part.
To do:
fill in the correct address of the key sections
such as text, data, bss.
tips: maybe you can find the answer in include/ .
look for the memory map.
*/
    = 0x80010000;
    .text : { *(.text) }
    .data : { *(.data) }
    .bss : { *(.bss) }

    end = . ;
    ~
    ~
    ~
    ~
    ~
    ~
    "tools/scse0_3.lds" 26L, 435C
}
```

Version v2.14.2- 发送文本到所有ssh终端 ...

Exercise 1.4

同上题，在内核布局中找到KSTACKTOP地址，li后jal跳转即可



```
.set mips2 /*.set is used to instruct how the assembler works
and control the order of instructions */
.set reorder

/* Disable interrupts */
mtc0 zero, CP0_STATUS

/* Disable watch exception. */
mtc0 zero, CP0_WATCHLO
mtc0 zero, CP0_WATCHHI

/* disable kernel mode cache */
mfc0 t0, CP0_CONFIG
and t0, ~0x7
ori t0, 0x2
mtc0 t0, CP0_CONFIG

/*
Exercise 1.4. Please complete this part.
To do:
set up stack.
hint: you can reference the memory layout in the include/mmio.h.
call main func.
*/
li sp, 0x80400000
jal main

loop:
j loop
nop
END(_start) /*the function defined in asm.h*/
```

Exercise 1.5

这题比较考验C语言基本功，理解了printf中每个参数的含义后逐个判断输入的字符即可完成part1，然后模仿其他case的思路即可完成part2整数的输出，要注意num是负数时要把negFlag置1后把num设为相反数，这点与其他case不同。

体会与感想

本次实验难度较Lab0稍有提升，但总体还算不太难，主要难点在于看懂别人写的代码和理解整个体系，自己动手写的代码量不大，难度也不高，在指导书和文件中仔细寻找都可以找到相关信息。但感觉做完之后我对操作系统启动部分的理解还不是很深刻，很多内容看完指导书之后就忘了。总体花了大约10h。

指导书反馈

```
/* we found a '%' */
/* check for long */
/* check for other prefixes */
/* check format flag */
```

printf实战中part1给的一部分注释比较容易迷惑人，发现%后应该先后对flag，width，precision，length进行检查，而注释中第一部分给的check for long不知所云，容易误导同学。

